

- 如果当链接器完成对命令行上输入文件的扫描后， U 是非空的，那么链接器就会输出一个错误并终止。否则，它会合并和重定位 E 中的目标文件，构建输出的可执行文件。

不幸的是，这种算法会导致一些令人困扰的链接时错误，因为命令行上的库和目标文件的顺序非常重要。在命令行中，如果定义一个符号的库出现在引用这个符号的目标文件之前，那么引用就不能被解析，链接会失败。比如，考虑下面的命令行发生了什么？

```
linux> gcc -static ./libvector.a main2.c
/tmp/cc9XH6Rp.o: In function 'main':
/tmp/cc9XH6Rp.o(.text+0x18): undefined reference to 'addvec'
```

在处理 `libvector.a` 时， U 是空的，所以没有 `libvector.a` 中的成员目标文件会添加到 E 中。因此，对 `addvec` 的引用是绝不会被解析的，所以链接器会产生一条错误信息并终止。

关于库的一般准则是将它们放在命令行的结尾。如果各个库的成员是相互独立的（也就是说没有成员引用另一个成员定义的符号），那么这些库就可以以任何顺序放置在命令行的结尾处。另一方面，如果库不是相互独立的，那么必须对它们排序，使得对于每个被存档文件的成员外部引用的符号 s ，在命令行中至少有一个 s 的定义是在对 s 的引用之后的。比如，假设 `foo.c` 调用 `libx.a` 和 `libz.a` 中的函数，而这两个库又调用 `liby.a` 中的函数。那么，在命令行中 `libx.a` 和 `libz.a` 必须处在 `liby.a` 之前：

```
linux> gcc foo.c libx.a libz.a liby.a
```

如果需要满足依赖需求，可以在命令行上重复库。比如，假设 `foo.c` 调用 `libx.a` 中的函数，该库又调用 `liby.a` 中的函数，而 `liby.a` 又调用 `libx.a` 中的函数。那么 `libx.a` 必须在命令行上重复出现：

```
linux> gcc foo.c libx.a liby.a libx.a
```

另一种方法是，我们可以将 `libx.a` 和 `liby.a` 合并成一个单独的存档文件。

 **练习题 7.3** a 和 b 表示当前目录中的目标模块或者静态库，而 $a \rightarrow b$ 表示 a 依赖于 b ，也就是说 b 定义了一个被 a 引用的符号。对于下面每种场景，请给出最小的命令行（即一个含有最少数量的目标文件和库参数的命令），使得静态链接器能解析所有的符号引用。

- $p.o \rightarrow libx.a$
- $p.o \rightarrow libx.a \rightarrow liby.a$
- $p.o \rightarrow libx.a \rightarrow liby.a$ 且 $liby.a \rightarrow libx.a \rightarrow p.o$

7.7 重定位

一旦链接器完成了符号解析这一步，就把代码中的每个符号引用和正好一个符号定义（即它的一个输入目标模块中的一个符号表条目）关联起来。此时，链接器就知道它的输入目标模块中的代码节和数据节的确切大小。现在就可以开始重定位步骤了，在这个步骤中，将合并输入模块，并为每个符号分配运行时地址。重定位由两步组成：

- 重定位节和符号定义。在这一步中，链接器将所有相同类型的节合并为同一类型的新的聚合节。例如，来自所有输入模块的 `.data` 节被全部合并成一个节，这个节成为输出的可执行目标文件的 `.data` 节。然后，链接器将运行时内存地址赋给新的聚合节，赋给输入模块定义的每个节，以及赋给输入模块定义的每个符号。当这一步完成时，程序中的每条指令和全局变量都有唯一的运行时内存地址了。

- 重定位节中的符号引用。在这一步中，链接器修改代码节和数据节中对每个符号的引用，使得它们指向正确的运行时地址。要执行这一步，链接器依赖于可重定位目标模块中称为重定位条目(relocation entry)的数据结构，我们接下来将会描述这种数据结构。

7.7.1 重定位条目

当汇编器生成一个目标模块时，它并不知道数据和代码最终将放在内存中的什么位置。它也不知道这个模块引用的任何外部定义的函数或者全局变量的位置。所以，无论何时汇编器遇到对最终位置未知的目标引用，它就会生成一个重定位条目，告诉链接器在将目标文件合并成可执行文件时如何修改这个引用。代码的重定位条目放在`.rel.text`中。已初始化数据的重定位条目放在`.rel.data`中。

图 7-9 展示了 ELF 重定位条目的格式。`offset` 是需要被修改的引用的节偏移。`symbol` 标识被修改引用应该指向的符号。`type` 告知链接器如何修改新的引用。`addend` 是一个有符号常数，一些类型的重定位要使用它对被修改引用的值做偏移调整。

```

code/link/elfstructs.c
1  typedef struct {
2      long offset;    /* Offset of the reference to relocate */
3      long type:32,   /* Relocation type */
4          symbol:32; /* Symbol table index */
5      long addend;    /* Constant part of relocation expression */
6  } Elf64_Rela;
code/link/elfstructs.c

```

图 7-9 ELF 重定位条目。每个条目表示一个必须被重定位的引用，并指明如何计算被修改的引用

ELF 定义了 32 种不同的重定位类型，有些相当隐秘。我们只关心其中两种最基本的重定位类型：

- `R_X86_64_PC32`。重定位一个使用 32 位 PC 相对地址的引用。回想一下 3.6.3 节，一个 PC 相对地址就是距程序计数器(PC)的当前运行时值的偏移量。当 CPU 执行一条使用 PC 相对寻址的指令时，它就将在指令中编码的 32 位值加上 PC 的当前运行时值，得到有效地址(如 `call` 指令的目标)，PC 值通常是下一条指令在内存中的地址。
- `R_X86_64_32`。重定位一个使用 32 位绝对地址的引用。通过绝对寻址，CPU 直接使用在指令中编码的 32 位值作为有效地址，不需要进一步修改。

这两种重定位类型支持 x86-64 小型代码模型(`small code model`)，该模型假设可执行目标文件中的代码和数据的总体大小小于 2GB，因此在运行时可以用 32 位 PC 相对地址来访问。GCC 默认使用小型代码模型。大于 2GB 的程序可以用 `-mcmodel=medium`(中型代码模型)和 `-mcmodel=large`(大型代码模型)标志来编译，不过在此我们不讨论这些模型。

7.7.2 重定位符号引用

图 7-10 展示了链接器的重定位算法的伪代码。第 1 行和第 2 行在每个节 `s` 以及与每个节相关联的重定位条目 `r` 上迭代执行。为了使描述具体化，假设每个节 `s` 是一个字节数组，每个重定位条目 `r` 是一个类型为 `Elf64_Rela` 的结构，如图 7-9 中的定义。另外，还

假设当算法运行时，链接器已经为每个节(用 `ADDR(s)` 表示)和每个符号都选择了运行时地址(用 `ADDR(r.symbol)` 表示)。第 3 行计算的是需要被重定位的 4 字节引用的数组 `s` 中的地址。如果这个引用使用的是 PC 相对寻址，那么它就用第 5~9 行来重定位。如果该引用使用的是绝对寻址，它通过第 11~13 行来重定位。

```

1  foreach section s {
2      foreach relocation entry r {
3          refptr = s + r.offset; /* ptr to reference to be relocated */
4
5          /* Relocate a PC-relative reference */
6          if (r.type == R_X86_64_PC32) {
7              refaddr = ADDR(s) + r.offset; /* ref's run-time address */
8              *refptr = (unsigned) (ADDR(r.symbol) + r.addend - refaddr);
9          }
10
11         /* Relocate an absolute reference */
12         if (r.type == R_X86_64_32)
13             *refptr = (unsigned) (ADDR(r.symbol) + r.addend);
14     }
15 }

```

图 7-10 重定位算法

让我们来看看链接器如何用这个算法来重定位图 7-1 示例程序中的引用。图 7-11 给出了(用 `objdump-dx main.o` 产生的)GNU OBJDUMP 工具产生的 `main.o` 的反汇编代码。

```

code/link/main-reloc
1  0000000000000000 <main>:
2      0:  48 83 ec 08          sub    $0x8,%rsp
3      4:  be 02 00 00 00      mov    $0x2,%esi
4      9:  bf 00 00 00 00      mov    $0x0,%edi    %edi = &array
5                      a: R_X86_64_32 array    Relocation entry
6
7      e:  e8 00 00 00 00      callq 13 <main+0x13> sum()
8                      f: R_X86_64_PC32 sum-0x4    Relocation entry
9      13: 48 83 c4 08          add    $0x8,%rsp
10     17: c3                  retq
code/link/main-reloc

```

图 7-11 `main.o` 的代码和重定位条目。原始 C 代码在图 7-1 中

`main` 函数引用了两个全局符号：`array` 和 `sum`。为每个引用，汇编器产生一个重定位条目，显示在引用的后面一行上。^①这些重定位条目告诉链接器对 `sum` 的引用要使用 32 位 PC 相对地址进行重定位，而对 `array` 的引用要使用 32 位绝对地址进行重定位。接下来两节会详细介绍链接器是如何重定位这些引用的。

1. 重定位 PC 相对引用

图 7-11 的第 6 行中，函数 `main` 调用 `sum` 函数，`sum` 函数是在模块 `sum.o` 中定义的。

① 回想一下，重定位条目和指令实际上存放在目标文件的不同节中。为了方便，OBJDUMP 工具把它们显示在一起。

call 指令开始于节偏移 0xe 的地方, 包括 1 字节的操作码 0xe8, 后面跟着的是对目标 sum 的 32 位 PC 相对引用的占位符。

相应的重定位条目 r 由 4 个字段组成:

```
r.offset = 0xf
r.symbol = sum
r.type   = R_X86_64_PC32
r.addend = -4
```

这些字段告诉链接器修改开始于偏移量 0xf 处的 32 位 PC 相对引用, 这样在运行时它会指向 sum 例程。现在, 假设链接器已经确定

```
ADDR(s) = ADDR(.text) = 0x4004d0
```

和

```
ADDR(r.symbol) = ADDR(sum) = 0x4004e8
```

使用图 7-10 中的算法, 链接器首先计算出引用的运行时地址(第 7 行):

```
refaddr = ADDR(s) + r.offset
          = 0x4004d0 + 0xf
          = 0x4004df
```

然后, 更新该引用, 使得它在运行时指向 sum 程序(第 8 行):

```
*refptr = (unsigned) (ADDR(r.symbol) + r.addend - refaddr)
          = (unsigned) (0x4004e8 + (-4) - 0x4004df)
          = (unsigned) (0x5)
```

在得到的可执行目标文件中, call 指令有如下的重定位的形式:

```
4004de: e8 05 00 00 00      callq 4004e8 <sum>      sum()
```

在运行时, call 指令将存放在地址 0x4004de 处。当 CPU 执行 call 指令时, PC 的值为 0x4004e3, 即紧随在 call 指令之后的指令的地址。为了执行这条指令, CPU 执行以下的步骤:

- 1) 将 PC 压入栈中
- 2) $PC \leftarrow PC + 0x5 = 0x4004e3 + 0x5 = 0x4004e8$

因此, 要执行的下一条指令就是 sum 例程的第一条指令, 这当然就是我们想要的!

2. 重定位绝对引用

重定位绝对引用相当简单。例如, 图 7-11 的第 4 行中, mov 指令将 array 的地址(一个 32 位立即数值)复制到寄存器 %edi 中。mov 指令开始于节偏移量 0x9 的位置, 包括 1 字节操作码 0xbf, 后面跟着对 array 的 32 位绝对引用的占位符。

对应的占位符条目 r 包括 4 个字段:

```
r.offset = 0xa
r.symbol = array
r.type   = R_X86_64_32
r.addend = 0
```

这些字段告诉链接器要修改从偏移量 0xa 开始的绝对引用, 这样在运行时它将会指向 array 的第一个字节。现在, 假设链接器已经确定

```
ADDR(r.symbol) = ADDR(array) = 0x601018
```


链接器使用图 7-10 中算法的第 13 行修改了引用：

```
*refptr = (unsigned) (ADDR(r.symbol) + r.addend)
          = (unsigned) (0x601018 + 0)
          = (unsigned) (0x601018)
```

在得到的可执行目标文件中，该引用有下面的重定位形式：

```
4004d9: bf 18 10 60 00      mov     $0x601018,%edi    %edi = &array
```

综合到一起，图 7-12 给出了最终可执行目标文件中已重定位的 .text 节和 .data 节。在加载的时候，加载器会把这些节中的字节直接复制到内存，不再进行任何修改地执行这些指令。

1	00000000004004d0 <main>:		
2	4004d0: 48 83 ec 08	sub	\$0x8,%rsp
3	4004d4: be 02 00 00 00	mov	\$0x2,%esi
4	4004d9: bf 18 10 60 00	mov	\$0x601018,%edi %edi = &array
5	4004de: e8 05 00 00 00	callq	4004e8 <sum> sum()
6	4004e3: 48 83 c4 08	add	\$0x8,%rsp
7	4004e7: c3	retq	
8	00000000004004e8 <sum>:		
9	4004e8: b8 00 00 00 00	mov	\$0x0,%eax
10	4004ed: ba 00 00 00 00	mov	\$0x0,%edx
11	4004f2: eb 09	jmp	4004fd <sum+0x15>
12	4004f4: 48 63 ca	movslq	%edx,%rcx
13	4004f7: 03 04 8f	add	(%rdi,%rcx,4),%eax
14	4004fa: 83 c2 01	add	\$0x1,%edx
15	4004fd: 39 f2	cmp	%esi,%edx
16	4004ff: 7c f3	j1	4004f4 <sum+0xc>
17	400501: f3 c3	repz retq	

a) 已重定位的 .text 节

1	0000000000601018 <array>:
2	601018: 01 00 00 00 02 00 00 00

b) 已重定位的 .data 节

图 7-12 可执行文件 prog 的已重定位的 .text 节和 .data 节。原始的 C 代码在图 7-1 中



练习题 7.4 本题是关于图 7-12a 中的已重定位程序的。

- 第 5 行中对 sum 的重定位引用的十六进制地址是多少？
- 第 5 行中对 sum 的重定位引用的十六进制值是多少？



练习题 7.5 考虑目标文件 m.o 中对 swap 函数的调用(图 7-5)。

```
9: e8 00 00 00 00      callq e <main+0xe>    swap()
```

它的重定位条目如下：

```
r.offset = 0xa
r.symbol = swap
r.type   = R_X86_64_PC32
r.addend = -4
```

现在假设链接器将 m.o 中的 .text 重定位到地址 0x4004d0，将 swap 重定位到地址

0x4004e8。那么 `callq` 指令中对 `swap` 的重定位引用的值是什么？

7.8 可执行目标文件

我们已经看到链接器如何将多个目标文件合并成一个可执行目标文件。我们的示例 C 程序，开始时是一组 ASCII 文本文件，现在已经被转化为一个二进制文件，且这个二进制文件包含加载程序到内存并运行它所需的所有信息。图 7-13 概括了一个典型的 ELF 可执行文件中的各类信息。

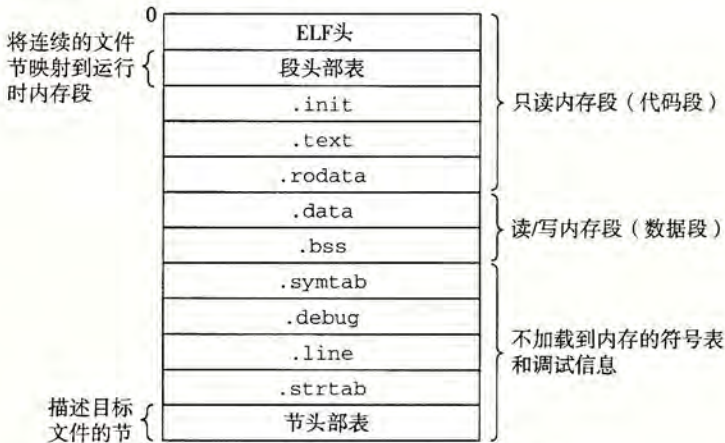


图 7-13 典型的 ELF 可执行目标文件

可执行目标文件的格式类似于可重定位目标文件的格式。ELF 头描述文件的总体格式。它还包括程序的入口点(entry point)，也就是当程序运行时要执行的第一条指令的地址。`.text`、`.rodata` 和 `.data` 节与可重定位目标文件中的节是相似的，除了这些节已经被重定位到它们最终的运行时内存地址以外。`.init` 节定义了一个小函数，叫做 `_init`，程序的初始化代码会调用它。因为可执行文件是完全链接的(已被重定位)，所以它不再需要 `.rel` 节。

ELF 可执行文件被设计得很容易加载到内存，可执行文件的连续的片(chunk)被映射到连续的内存段。程序头部表(program header table)描述了这种映射关系。图 7-14 展示了可执行文件 `prog` 的程序头部表，是由 `OBJDUMP` 显示的。

```

code/link/prog-exe.d
Read-only code segment
1 LOAD off 0x0000000000000000 vaddr 0x0000000000400000 paddr 0x0000000000400000 align 2**21
2 filesz 0x0000000000000069c memsz 0x0000000000000069c flags r-x

Read/write data segment
3 LOAD off 0x0000000000000df8 vaddr 0x0000000000600df8 paddr 0x0000000000600df8 align 2**21
4 filesz 0x0000000000000228 memsz 0x0000000000000230 flags rw-

code/link/prog-exe.d

```

图 7-14 示例可执行文件 `prog` 的程序头部表

off: 目标文件中的偏移; vaddr/paddr: 内存地址; align: 对齐要求; filesz: 目标文件中的段大小; memsz: 内存中的段大小; flags: 运行时访问权限。

从程序头部表，我们会看到根据可执行目标文件的内容初始化两个内存段。第 1 行和